

1. Brute force method

The screenshot shows a submission on NeetCode 150 for the "Group Anagrams" problem. The submission is marked as "Accepted" for 27 / 27 test cases. The runtime is 190ms (Beats 1.48%) and the memory is 8.5 MB (Beats 94.43%). A bar chart shows the runtime distribution, with the submission's runtime (190ms) being significantly higher than the majority of other submissions. The code is a Python class solution that uses a nested loop to compare strings and a set to track visited indices.

Question | Solution | Submissions | Discuss (185)

← All Submissions

Accepted 27 / 27 test cases

Memory: 8.5 MB · Time: 190ms · Submitted at: 05/28/2026 12:50 · 🔄

Runtime 190ms | Beats 1.48%

Memory 8.5 MB | Beats 94.43%

88%
66%
44%
22%
0%

48ms 432ms 848ms 1360ms 2480ms

Analyze Complexity

```
1 class Solution:
2     def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
3         result = []
4         visited = set()
5         for i in range(len(strs)):
6             if i in visited:
7                 continue
8             group = [strs[i]]
9             visited.add(i)
10            for j in range(i+1, len(strs)):
11                if sorted(strs[i]) == sorted(strs[j]):
12                    group.append(strs[j])
13                    visited.add(j)
14            result.append(group)
15        return result
16
```

Test Case | Output

Accepted Passed test cases: 27 / 27

Visualize code

Console | Run

2. Optimized method - hash map

The screenshot shows a submission on NeetCode 150 for the "Group Anagrams" problem using a hash map. The submission is marked as "Accepted" for 27 / 27 test cases. The runtime is 51ms (Beats 100.00%) and the memory is 8.5 MB (Beats 94.43%). A bar chart shows the runtime distribution, with the submission's runtime (51ms) being significantly lower than the majority of other submissions. The code is a Python class solution that uses a defaultdict to map sorted strings to their indices.

Question | Solution | Submissions | Discuss (185)

← All Submissions

Accepted 27 / 27 test cases

Memory: 8.5 MB · Time: 51ms · Submitted at: 05/28/2026 13:20 · 🔄

Runtime 51ms | Beats 100.00%

Memory 8.5 MB | Beats 94.43%

88%
66%
44%
22%
0%

48ms 432ms 848ms 1360ms 2480ms

Analyze Complexity

```
1 class Solution:
2     def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
3         result = defaultdict(list)
4         for word in strs:
5             count = [0] * 26
6             for ch in word:
7                 count[ord(ch) - ord('a')] += 1
8             result[tuple(count)].append(word)
9         return list(result.values())
```

Test Case | Output

Accepted Passed test cases: 27 / 27

Visualize code

You have successfully completed this problem!

Console | Run | Submit

🔥 Brute Force vs HashMap + Count Array

Feature	Brute Force	HashMap + Count Array
Main Idea	Compare every word with every other word	Create frequency key and
Technique	Nested loops	HashMap
Key Used	Sorted comparison	Frequency tuple
Nested Loops	✅ Yes	❌ No
Sorting Needed	✅ Yes	❌ No
Time Complexity	🔥 $O(n^2 * k \log k)$	🚀 $O(n * k)$
Space Complexity	$O(n)$	$O(n)$
Speed	❌ Slow	✅ Fast
Easy to Understand	✅ Very easy	⚠️ Medium
Easy to Code	✅	⚠️ Slightly longer
Best For	Learning basics	Optimal interviews
Interview Level	Beginner approach	Advanced optimized